

Agile Software Development and Scrum



SoftEng
<http://softeng.polito.it>

Version 1.2.0
© Marco Torchiano, Antonio Vetrò, 2025



Licensing Note



This work is licensed under the Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 International License.

To view a copy of this license, visit

<http://creativecommons.org/licenses/by-nc-nd/4.0/>.

You are free: to copy, distribute, display, and perform the work

Under the following conditions:



Attribution. You must attribute the work in the manner specified by the author or licensor.



Non-commercial. You may not use this work for commercial purposes.



No Derivative Works. You may not alter, transform, or build upon this work.

- For any reuse or distribution, you must make clear to others the license terms of this work.
- Any of these conditions can be waived if you get permission from the copyright holder.

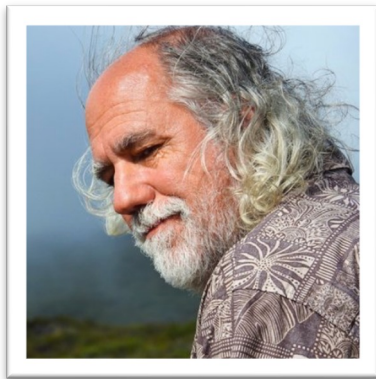
Your fair use and other rights are in no way affected by the above.

SOFTWARE + ENGINEERING

Software

..is the invisible language that
whispers stories of possibility
to the hardware

– Grady Booch



Engineering

The creative application of scientific principles to design or develop structures, machines, apparatus, or manufacturing processes, or works utilizing them singly or in combination; or to construct or operate the same with full cognizance of their design; or to forecast their behavior under specific operating conditions; all as respects an intended function, economics of operation and safety to life and property.

– ECPD

Engineering

- **Design**: the intentional solution to a problem within a set of constraints
- **Construction**: planning, monitoring, controlling the activities to achieve a solution, + tools and techniques
- **Operation**: conduction of the solution and adaptation, within its limits

Engineering

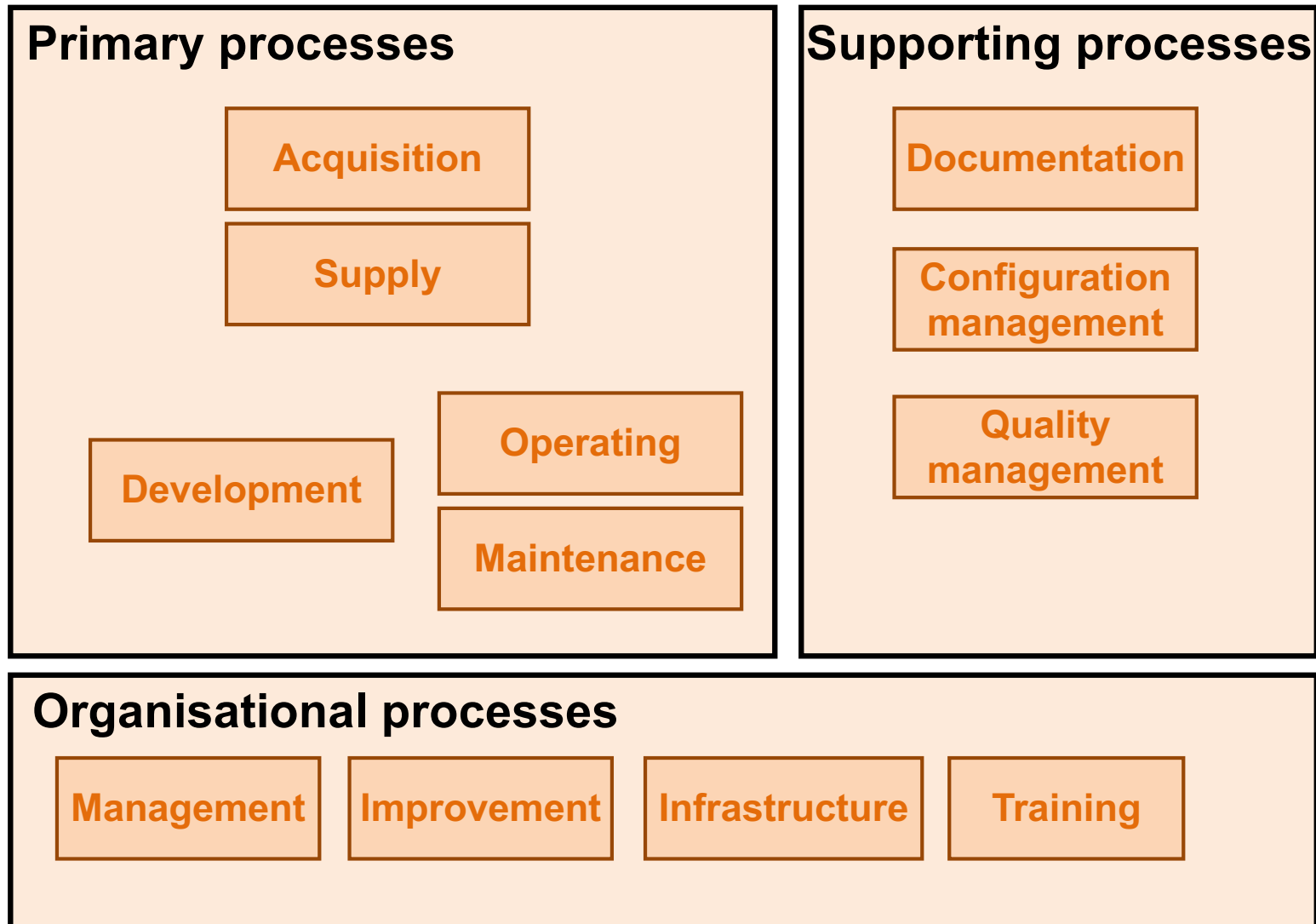
- **Design**: the intentional solution to a problem within a set of constraints
- **Construction**: planning, monitoring, controlling the activities to achieve a solution, + tools and techniques
- **Operation**: conduction of the solution and adaptation, within its limits

Construction

- How does Software Development work in practice?
 - ◆ Effort estimation
 - ◆ Planning development and release
 - ◆ Day-by-day practices
 - ◆ Customer relationship
 - ◆ Quality assurance
 - ◆ Acquisition
 - ◆ ...

Software life cycle processes

ISO/IEC 12207



AGILE SOFTWARE DEVELOPMENT

Agile Development



© Scott Adams, Inc./Dist. by UFS, Inc.

History

- Through the 90s several consultant experienced the limitation of “hard” waterfall-like processes
- Each developed his own flavor of agile methodology

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Agilemanifesto.org

Individuals and interactions *over*
processes and tools

Working software *over*
comprehensive documentation

Customer collaboration *over*
contract negotiation

Responding to change *over*
following a plan

Agile Manifesto – Principles

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.

Agile Manifesto – Principles

5. Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

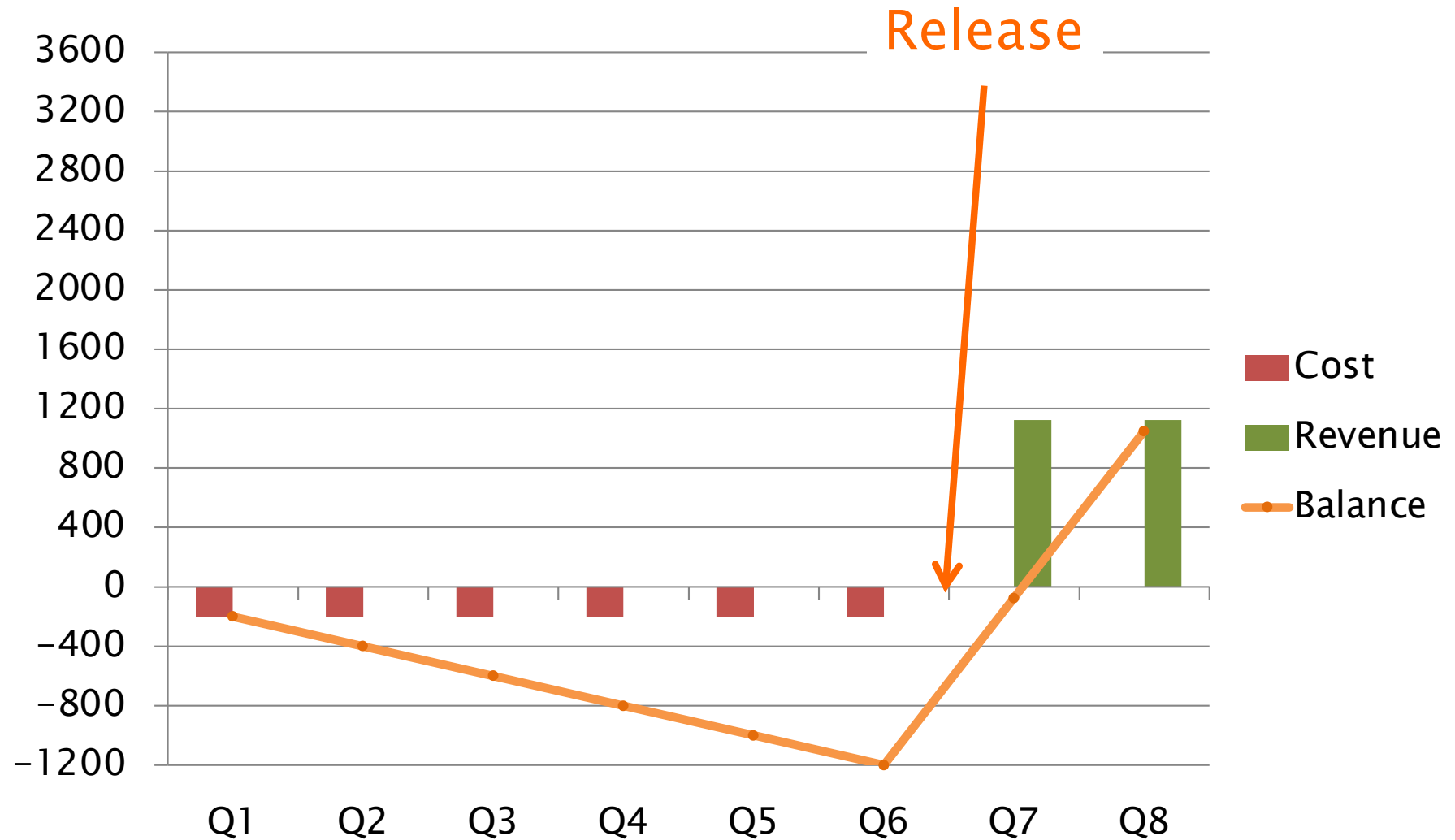
Agile Manifesto – Principles

- 9. Continuous attention to technical excellence and good design enhances agility.
- 10. Simplicity – the art of maximizing the amount of work not done – is essential.
- 11. The best architectures, requirements, and designs emerge from self-organizing teams.
- 12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Fictional Case

- Project characteristics
 - ◆ Duration: 1.5 years
 - ◆ Budget: 800 K€/year
 - ◆ Revenue: 4.5 M€/year
- Approaches
 - ◆ Waterfall
 - One release, at the end
 - ◆ Agile
 - 6 releases, each quarter

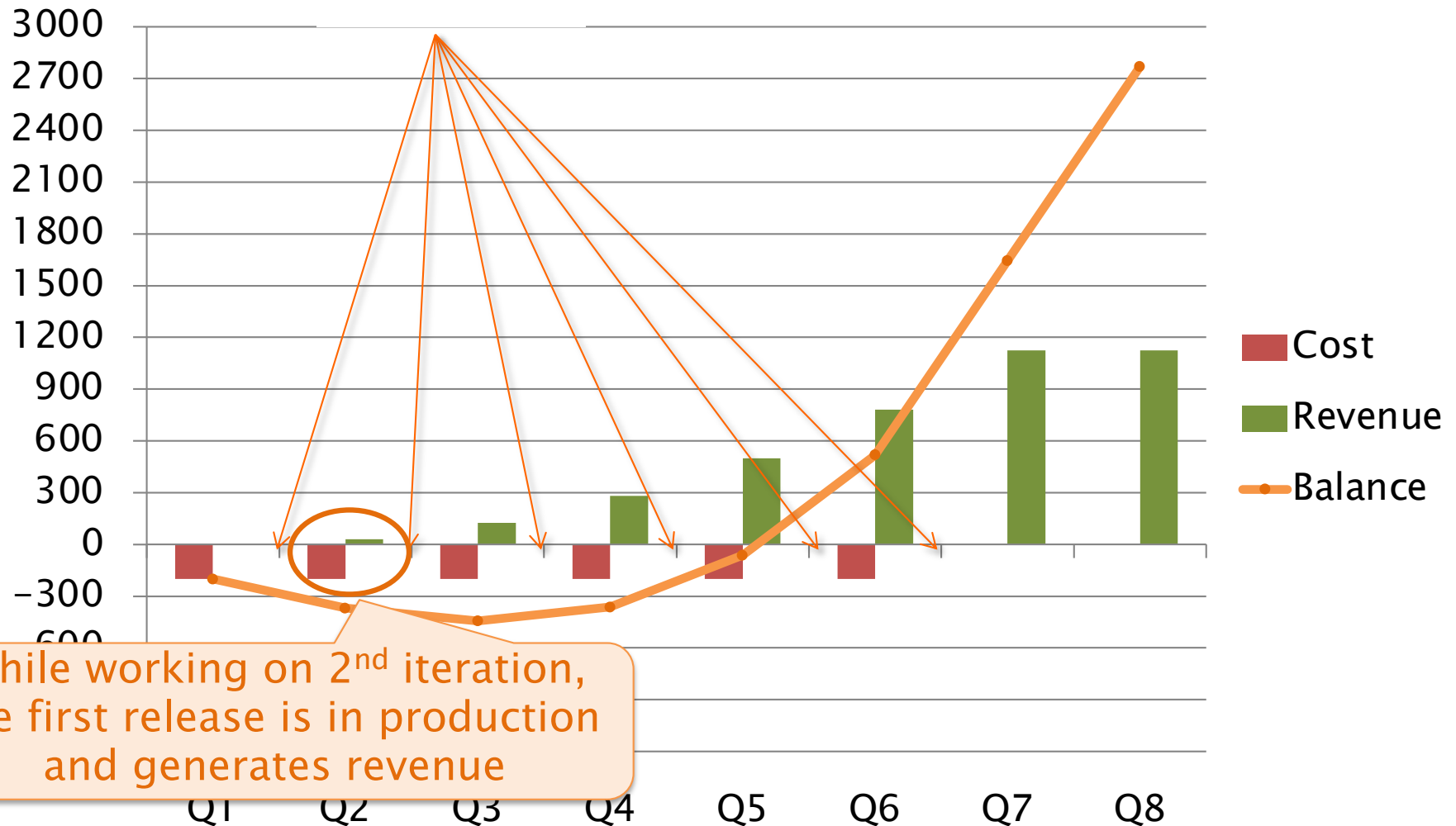
Waterfall



ROI: 88%

Agile / Iterative

Releases



While working on 2nd iteration, the first release is in production and generates revenue

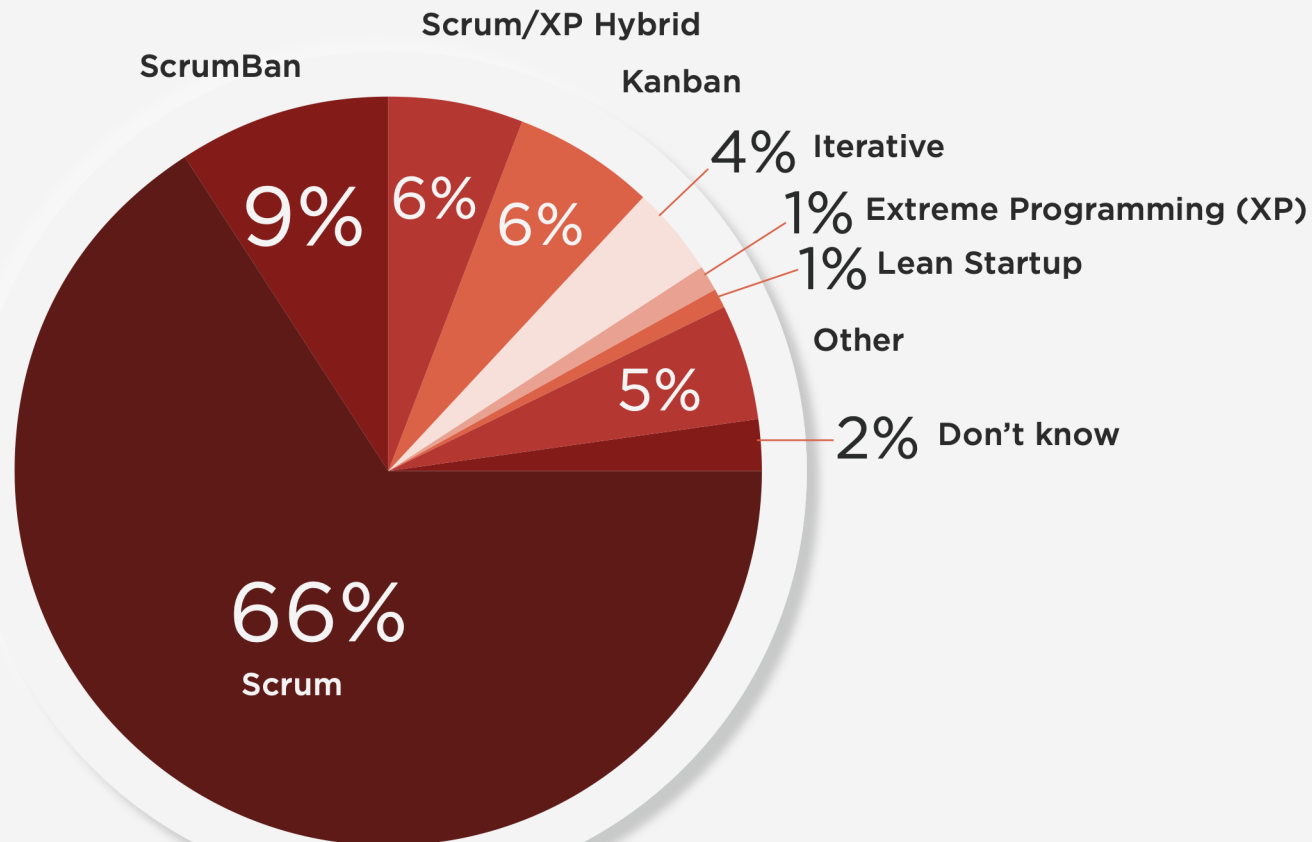
ROI: 231%

From manifesto to practice

- Tight collaboration between developers and stakeholder
 - ◆ over the entire course of the project
- Self-organizing teams
- Software SCRUM is the most used methodology
 - ◆ Other methodologies like Kanban and eXtreme Programming are less used

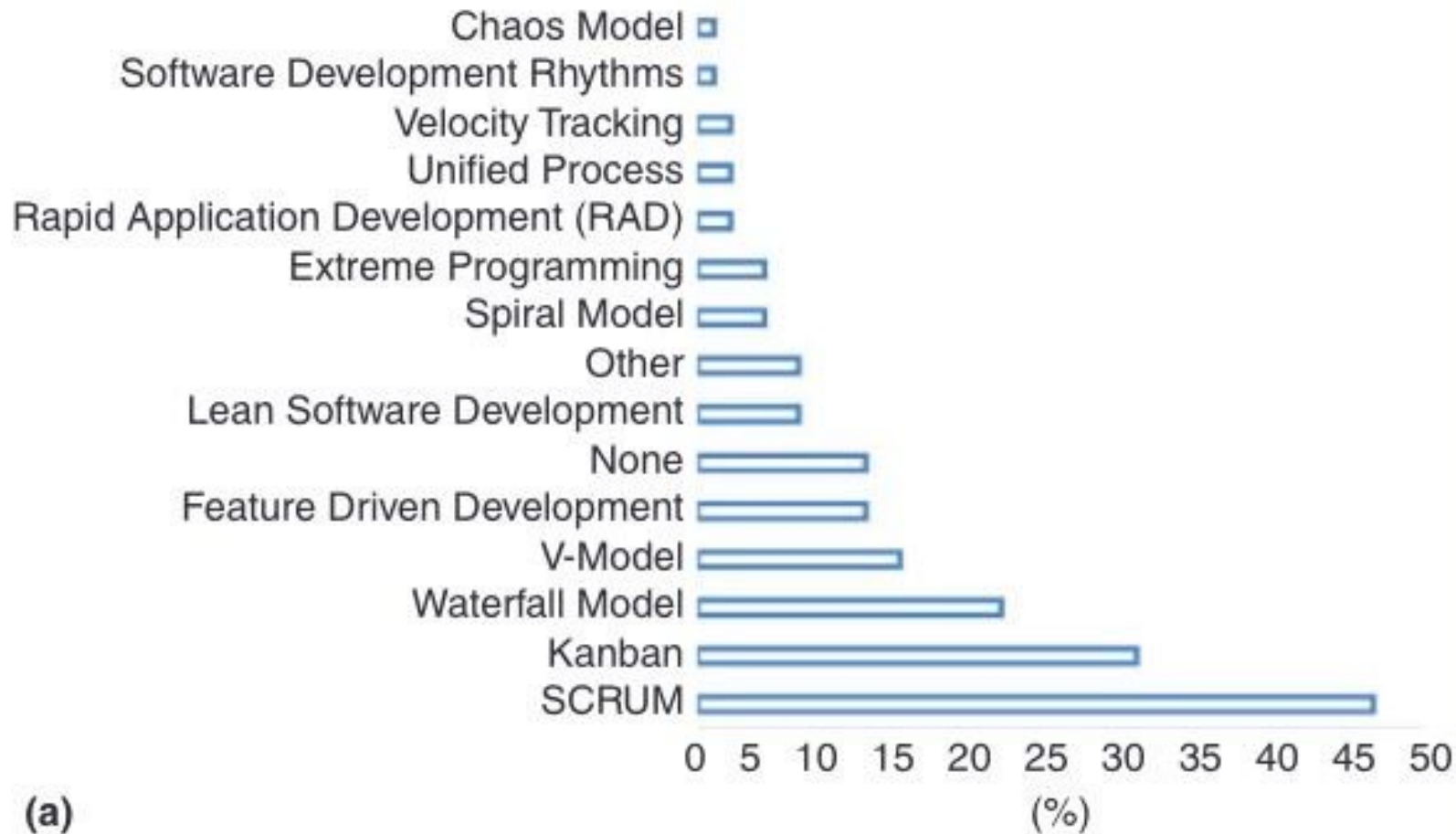
Diffusion of Agile Methods

Which Agile methodology do you follow most closely at the team level?



Source: 15th State of Agile Report,
available at: <https://explore.digital.ai/state-of-agile/15th-state-of-agile-report>

Diffusion (USA data)



M. Kassab and P. Laplante, "The Current and Evolving Landscape of Requirements Engineering in Practice," IEEE Software, vol. 39, no. 5, pp. 76–83, Sep. 2022, doi: 10.1109/MS.2022.3147692.

Software Quality

- Quality is a key component in any engineering practice
- How does it work in Sw engineering?
 - ◆ Sw analytics allow measuring and controlling Sw projects
 - ◆ Code smells indicate possible problems
 - ◆ Technical Debt

SCRUM

Scrum



Definition

- Scrum:
 - ◆ An ordered formation of players, used to restart play in rugby
 - ◆ A team-based framework to develop complex systems and products.

Scrum creators



Jeff Sutherland and Ken Schwaber

Picture from their article: «The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game» Nov. 2017

Principles

Agile Development Principles

- Test as you go
- Deliver product early and often
 - ◆ To get feedback
- Document as you go, only as required
- Build cross-functional teams

Theoretical foundations

- Empirical process control theory
- Iterative and incremental approach
 - ◆ to optimize predictability
 - ◆ to control risk

Scrum pillars

- Transparency
 - ◆ Process visible to whom is responsible
- Inspection
 - ◆ On artifacts and goals to detect variances
- Adaptation
 - ◆ To meet the goals

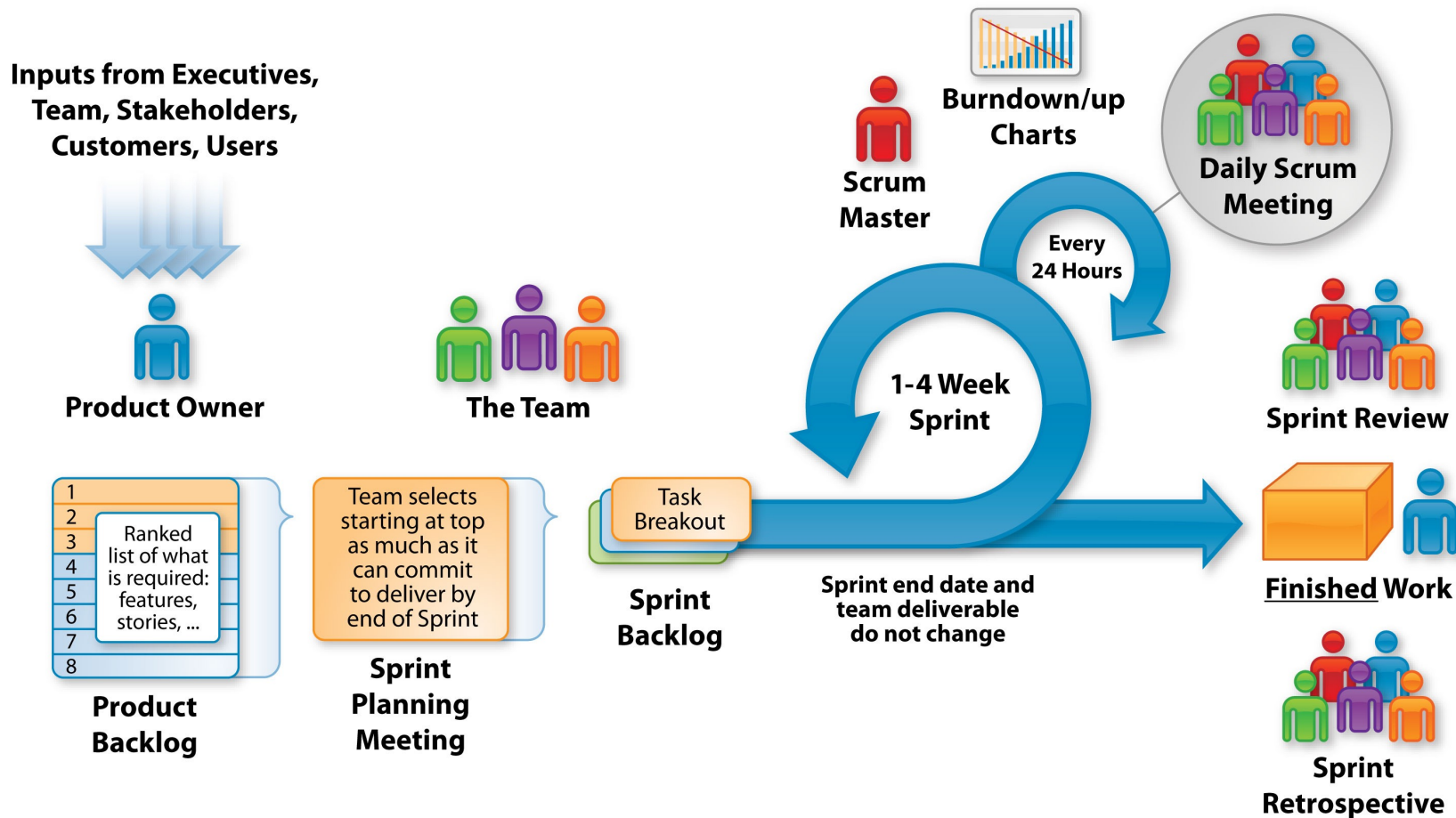
Adherence to pillars

- Four formal events:
 - ◆ Sprint planning
 - ◆ Daily scrum
 - ◆ Sprint review
 - ◆ Sprint retrospective

Scrum values

- Commitment
- Courage
- Focus
- Openness
- Respect

Scrum process in a nutshell



From: Rupert Dürre, «Analysis of User Stories and Effort Estimations in Agile Software Development» Technische Universität München

Roles

Scrum Roles

- Product Owner
- Scrum Master
- Team member

Product Owner

- Controls the priority order of items in the team's backlog
 - ◆ Directs team toward most valuable work
 - ◆ Work closely with the stakeholders in order to deliver the maximum business value
 - What needs to be built and when

Product Owner

- Makes sure that the needs of the customers and end-users are understood by the team
 - ◆ Requirements
 - ◆ Collaboration
 - Available to the team
- Keeper of the product vision
 - ◆ Who the product is built for
 - ◆ Why they need it
 - ◆ How they will use it

Product Owner

- Holds the vision for the product
- Represents the business
- Represents the customers
- Owns the product backlog
- Prioritizes stories
- Creates acceptance criteria for stories
- Is available to answer team members' questions

Scrum Master

- Acts as a coach
 - ◆ His goal is to produce a self-organizing team
 - ◆ Is a facilitator, NOT a boss
 - Good shepherd
 - Champion
 - Guardian
 - Scrum expert

Scrum Master

- Adjust to the team experience
 - ◆ As the team becomes self-managing the master steps back
- Remove impediments for the team
 - ◆ External
 - Escalates issues
 - Help solving resource issues (e.g. hw)
 - ◆ Internal
 - Helps the team see the problem
 - Encourages the team find a solution

Scrum Master

- Scrum expert and advisor
- Coach
- Impediment *bulldozer*
- Facilitator

Team Member

- The team is self-organizing
 - ◆ Tools
 - ◆ Techniques
 - ◆ Task assignment
- Team member estimate the effort for the features
- Team size: typically 5 to 9
 - ◆ Enough skill variety
 - ◆ Limited communication overhead

Team Member

- Cooperate to achieve the goal
- Willing to work outside his comfort zone when the team needs it
- The Scrum Team
 - ◆ P.Owner + S.Master + Team Members
- The Team
 - ◆ Scrum team except P.Owner and S.Master

Events

Sprint

- The basic iteration in the Scrum approach
- Produces a piece of working software to be demonstrated and reviewed at the end of the sprint
- Length
 - ◆ From 1 to 4 weeks long

Sprint organization (1 week)

	Monday	Tuesday	Wednesday	Thursday	Friday
Sprint Cycle	Sprint Planning (2h)	Scrum (15m)	Scrum (15m)	Scrum (15m)	Scrum (15m)
					Sprint Review (1 / 2 h)
				Story Time (1h)	Retrospective (1h)

Sprint Planning

[illegible]

- 1. What will we do?
 - ◆ Set of committed stories
 - ◆ Product owner proposes story
 - ◆ Team members decide whether commit
 - Team's velocity / Yesterday's weather
- 2. How will we do it?
 - ◆ Decompose stories into tasks
 - May trigger renegotiation of stories
 - ◆ Max effort per task: half-day

Sprint Backlog

[illegible]

- List of stories
- Related tasks
- Estimation:
 - ◆ Task hours
 - ◆ Task points
 - ◆ Task count

Effort Estimation

- Essential to define a predictable schedule
- Relative vs. Absolute estimates
 - ◆ Former are much easier
 - ◆ Stories are sized in term of “points”
 - Using Fibonacci series
 - ◆ After a sprint is complete an estimate of stories per sprint i.e. “velocity” is done
 - Velocity is NOT a performance measure

Estimation Poker

- Every team member pick a card and show it at the same time as the others
- High and low discuss the rationale
- Consensus is achieved
 - ♦ It is possible to get feedback from the product owner during the process



Estimation Poker

- Every team member pick a card and show it at the same time as the others
- Then each one explain the rationale
- Consensus is achieved with feedback from the product owner



«Estimation wall»

1: Estimation Categories with assigned user stories

2: Definition of the estimation unit

3: Explanation of the est. process

4: Retrospective and best practices

Rupert Dürre,
op. cit..

Daily Scrum

[illegible]

- Daily
 - ◆ Any time suitable to the team
- Small
 - ◆ Only members of the dev team
- Brief
 - ◆ Keep updated, standing, 15 min max
- Pointed
 - ◆ What has been done
 - ◆ What will be done
 - ◆ Current obstacles

[illegible]

- SoftEng**
<http://softeng.polito.it>

Retrospective

[illegible]

- Set the stage
- Gather data
 - ◆ What happened?
 - ◆ Time-line and artifacts
- Generate insights
 - ◆ Understand causes, no finger pointing
- Decide what to do
 - ◆ One team-improvement task for next sprint, under the team's control
- Close
 - ◆ Appreciation exercise

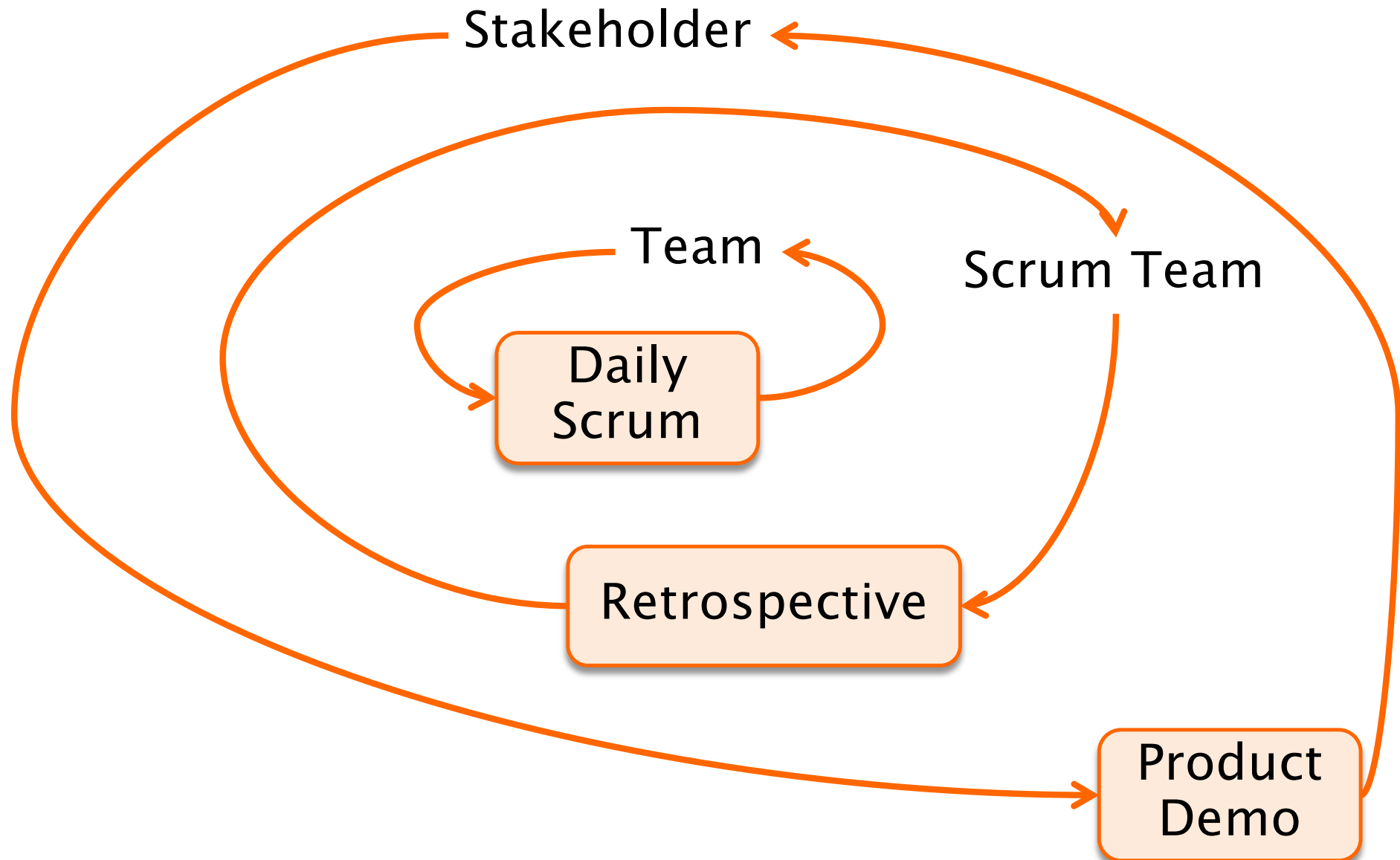
Esther Derby and Diana Larsen,
“Agile Retrospectives: Making Good Teams Great”

[illegible]

- “At the end of a project everyone knows so much more. Naturally we will discover decisions and actions we wish we could do over. This is wisdom to be celebrated...”*

SoftEng
<http://softeng.polito.it>

Inspect and Adapt

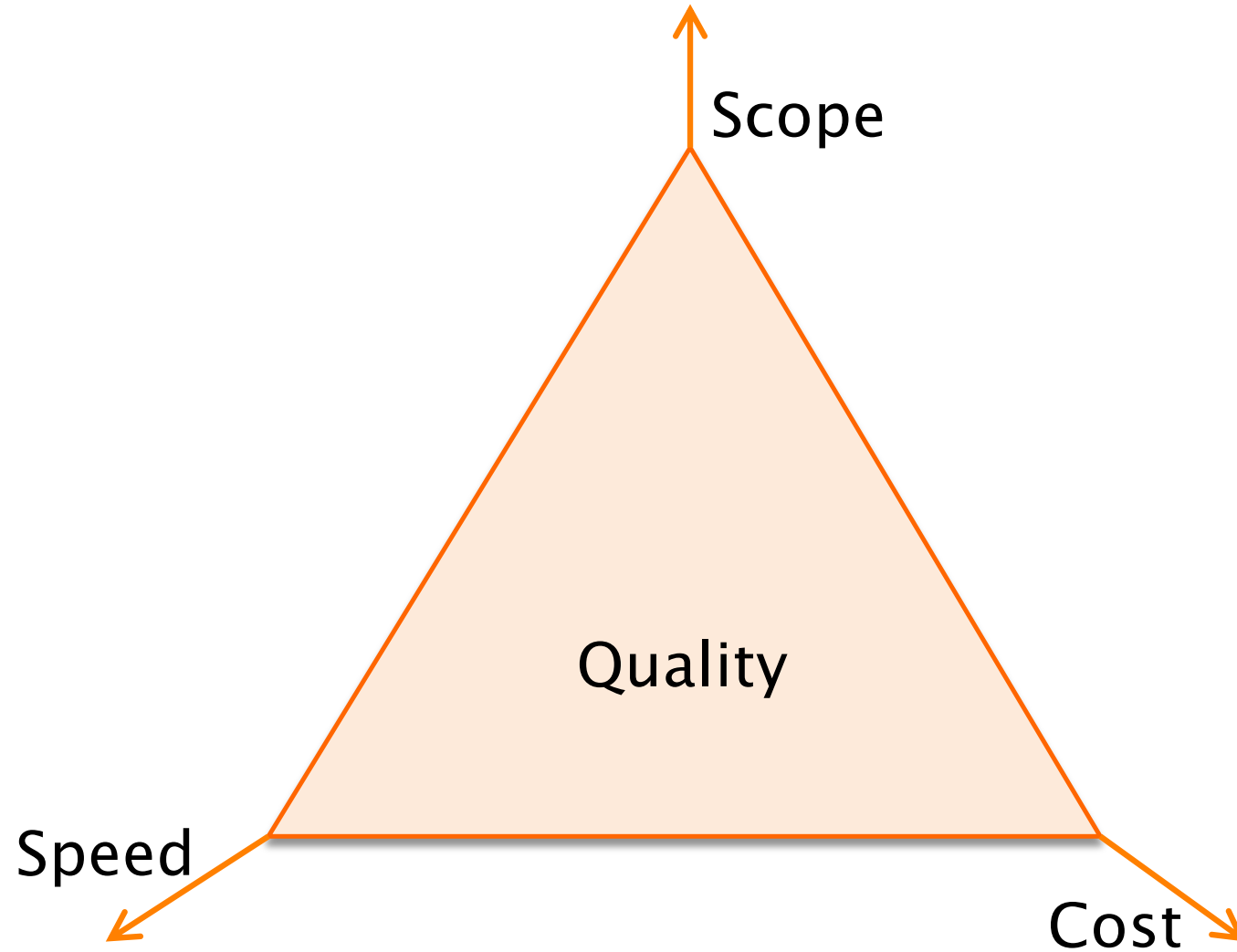


Release (long term) Planning

- Fixed Scope
- Fixed Date
- Fixed Scope & Fixed Date
 - ◆ Scrum is not a



Iron Triangle



Release (long term) Planning

Adding manpower to a late software project makes it later

Fred Brooks,
“The Mythical Man-Month”

Artifacts

Artifacts

- Product Backlog
 - ◆ Includes Product Backlog Items (PBI)
 - ◆ We will use User Stories as PBIs
- Sprint Backlog
 - ◆ Detailed small tasks

Product Backlog

- Anything that *will* consume team resources
- Ever changing
- Strict ordering of the stories
- Property of the Product Owner
- Can be a spreadsheet or a wall

Example

Story	Story points
As an <i>unregistered user</i> I want <i>to create a new account</i> So that <i>I can buy items</i>	3
As a <i>customer</i> I want <i>to use my Card</i> So that <i>I can buy the items</i>	8
As a <i>user</i> I want <i>to add items</i> <i>to my wish list</i>	5

Forecasted velocity: 12 pt per sprint
Sprint length: 2 weeks

TOT 16

Product Backlog Items

- A.k.a. stories, must be:
 - ♦ Independent
 - ♦ Negotiable
 - ♦ Valuable
 - ♦ Estimable
 - ♦ Small
 - ♦ Testable

“INVEST”

Product Backlog

- The backlog must be “DEEP”
 - ◆ Detailed appropriately
 - ◆ Estimated
 - ◆ Emergent
 - ◆ Prioritized

Story

- Essential description of a desired functionality from the point of view of a system user
- Acceptance criteria
 - ◆ Written by PO
 - ◆ Easy to turn into (automated) tests

Story

- Written following the template:

As a *<actor type>*

I want *<to do something>*

So that *<some value is created>*

Story – example

As an unregistered user
I want to create a new account
So that I can buy items

Story – example

As a customer
I want to use my Card
So that I can buy the items

Story – example

As a user
I want to add items
~~So that~~ *to my wish list*

Story – example

As a user
I want to delete items
~~So that~~ *from my wish list*

Story – example

As an administrator
I want to list history of purchases
So that I can compute statistics

Story – example

As an *administrator*
I want *to insert items in the website*
So that *I can sell them*

Story – example

As an administrator
I want to modify the price of existing items
So that

Story – example

As an administrator

*I want to receive an alert of low quantity
of items in the inventory*

*So that I can decide whether to send a
new order*

Story – example

As an inventory user
I want to modify the quantity of items
So that I can update websites with new orders

Aspects of a user story 1 / 3

- Card
 - ◆ Essential description
 - ◆ Extended with notes for estimation
 - ◆ “Promise for a conversation”

Aspects of a user story 2/3

- Conversation
 - ◆ Involves
 - the development team
 - the product owner
 - the customer
 - other stakeholders
 - ◆ Several times
 - During estimation
 - When selected for implementation
 - When divided in subtasks
 - During implementation
 - In retrospectives

Aspects of a user story 3 / 3

- Confirmation
 - ◆ Conditions of satisfaction
 - ◆ Defined through conversation
 - ◆ Recorded as acceptance tests
 - ◆ Complementary to “definition of done”

Stories and Epics

- Story
 - ◆ Manageable piece of requirement
 - It is INVEST
- Epic
 - ◆ Too large a story
 - ◆ Needs to be split into stories
 - ◆ Often used a container for several stories

Epic – example

As a customer
I want to be able to view and modify
wish-list
So that I can decide later which items to
buy

Sprint Backlog

- Committed stories
 - ◆ Relative tasks (may change during sprint)
- Additional tasks, for example:
 - ◆ Team improvement
 - ◆ Research work
 - ◆ Performance and security requirements
 - ◆ Bug fixing
- Frozen at the end of sprint planning
- Property of the team

Additional tasks – examples

Diagnose and fix the order processing script error (issue#8367291)

Additional tasks – examples

Investigate solutions for speeding up updating the inventory system after a purchase

Additional tasks – examples

Reduce complexity of Class
OrderDiscountManager

Information Radiators

- Burn down chart
 - ◆ Residual work units
 - ◆ Sprint (team)
 - ◆ Release (PO)
- Burn up chart
 - ◆ Completed work units
- Task board
 - ◆ To do, In progress, Done

Task board – example

To do

Story #12

Story #7

Story #33

Story #34

Story #22

In progress

Story #4

Story #2

Done

Story #28

Story #1

Story #3

Definition of Done

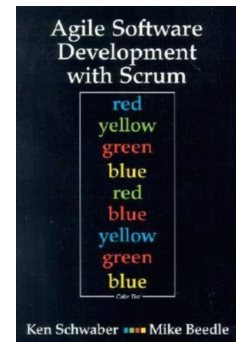
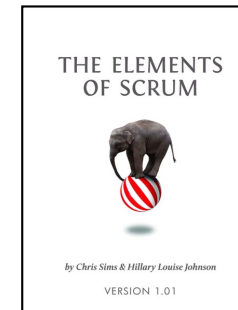
- Team shared definition
- Typically: shippable
 - ◆ Code review
 - ◆ Design review
 - ◆ Refactoring
 - ◆ Performance testing
 - ◆ Unit tests passed

Overview of main artifacts, roles and activities

		Participants				Artifacts & Activities	
		Product owner	Scrum Master	Devel. Team	Stakeholder	Used Artifacts	Activities
Events	Sprint Planning	X	X	X	(opt)	Product backlog Sprint backlog	Definition of a sprint goal Selection of user stories Definition of tasks
	Daily Scrum	(opt)	(opt)	X	(opt)	Sprint backlog Burn-Down/up chart	Update each other
	Story time	(opt)	X	X	(opt)	Product backlog	Backlog refinement Estimation of future stories
	Sprint review	X	X	X	X	Software Sprint backlog	Show piece of working software Collect feedback
	Sprint Retrospective	(opt)	X	X	(opt)	What is deemed useful/necessary	Examination of the last sprint in order to identify possible improvements

Suggested Readings

- K. Schwaber and J. Sutherland, The Scrum Guide, 2017 <http://www.scrumguides.org>
- K. Schwaber, M. Beedle. Agile Software Development with Scrum, Prentice-Hall, 2001
- C. Sims, H.L. Johnson. "The Elements of Scrum" DYNAMICON, 2011.
- Jeff Sutherland's [Scrum Handbook](#)
- H. Takeuchi and I. Nonaka, "The New New Product Development Game," Harvard Business Review, 1986.



Suggested readings

- R.Jeffries "The Nature of Software Development" The Pragmatic Programmers
- K.Schwaber, M.Beedle.
"Agile Software Development with Scrum", Prentice–Hall, 2001
- M.Cohn. "Succeeding with Agile", Addison–Wesley, 2010.
- C.Sims, H.L.Johnson. "The Elements of Scrum"
DYMEXICON, 2011.
- K.Schwaber and J.Sutherland "The Scrum Guide"
(<http://www.scrumguides.org>)
- D.Spinellis. Effective Debugging Addison–Wesley, 2016.
- P.Grubb, A.Takang "Software Maintenance: concepts and practice" World Scientific Publishing, 2003